

LAPORAN UJIAN AKHIR SEMESTER
PROYEK AKHIR CLOUD COMPUTING

**IMPLEMENTASI APLIKASI MULTI-CONTAINER DENGAN
DOCKER, ORKESTRASI, DAN CI/CD**

Studi Kasus: Todo List Application (Laravel + MariaDB)

Disusun oleh:

Purna Siswantomo - C2C023160

Erifa Dwi Astuti - C2C023161

Kelas: [Kelas Anda]

Mata Kuliah: Komputasi Awan (Cloud Computing)

Tahun Akademik: 2025/2026

Repository: <https://github.com/Purna-Siswantomo/ToDo-docker>

DAFTAR ISI

BAB 1 PENDAHULUAN

- 1.1 Latar Belakang
- 1.2 Permasalahan
- 1.3 Tujuan
- 1.4 Manfaat

BAB 2 ANALISIS DAN ARSITEKTUR

- 2.1 Deskripsi Aplikasi
- 2.2 Pengguna Sistem
- 2.3 Teknologi yang Digunakan
- 2.4 Diagram Arsitektur
- 2.5 Fungsi Setiap Komponen

BAB 3 IMPLEMENTASI APLIKASI

- 3.1 Fitur Utama
- 3.2 Struktur Data
- 3.3 Implementasi Model dan Controller
- 3.4 Routing dan Endpoint
- 3.5 Konfigurasi Environment

BAB 4 IMPLEMENTASI CONTAINER

- 4.1 Dockerfile
- 4.2 Docker Compose
- 4.3 Network dan Komunikasi Antar Service
- 4.4 Persistent Volume
- 4.5 Health Check dan Ketahanan Layanan

BAB 5 IMPLEMENTASI CI/CD

- 5.1 Tujuan Pipeline
- 5.2 Alur Workflow
- 5.3 Automated Testing sebagai Quality Gate
- 5.4 Simulasi Pipeline Gagal dan Berhasil

BAB 6 PENGUJIAN

- 6.1 Pengujian Fungsional
- 6.2 Pengujian Automated Test
- 6.3 Pengujian Container dan Compose
- 6.4 Pengujian Persistent Volume
- 6.5 Pengujian Ketahanan Layanan (Health Check)

BAB 7 KESIMPULAN

- 7.1 Kesimpulan Umum
- 7.2 Kendala dan Solusi
- 7.3 Rencana Pengembangan

LAMPIRAN

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Perkembangan cloud computing mendorong kebutuhan akan aplikasi yang mudah dipindahkan, konsisten antar lingkungan, dan mudah dikelola. Salah satu pendekatan yang banyak digunakan untuk menjawab kebutuhan tersebut adalah containerization menggunakan Docker. Dengan Docker, sebuah aplikasi dapat dikemas bersama dependensinya ke dalam image yang sama, sehingga perilaku aplikasi menjadi lebih stabil ketika dijalankan pada mesin yang berbeda. Pada proyek ini, konsep tersebut diterapkan melalui pengembangan aplikasi Todo List berbasis Laravel yang dijalankan di dalam arsitektur multi-container.

1.2 Permasalahan

Sebelum menggunakan container, aplikasi sering mengalami perbedaan environment antara mesin pengembang, penguji, dan production. Perbedaan versi dependency, konfigurasi database, serta proses instalasi manual dapat menimbulkan error yang sulit dilacak. Selain itu, jika database dijalankan tanpa persistent volume, data berisiko hilang saat container dihentikan. Permasalahan lain adalah kurangnya otomasi dalam pengujian dan build, sehingga perubahan kode tidak selalu diverifikasi sebelum diterapkan. Hal-hal tersebut menjadi alasan mengapa pendekatan Docker, Docker Compose, volume, health check, dan CI/CD perlu diterapkan dalam proyek ini.

1.3 Tujuan

Tujuan proyek ini adalah membangun aplikasi sederhana yang berjalan dalam arsitektur multi-container dengan container aplikasi dan container database. Proyek ini juga bertujuan untuk menerapkan persistent volume agar data tetap aman, menggunakan environment variable untuk konfigurasi yang fleksibel, menambahkan health check untuk pemantauan layanan, serta menerapkan automated testing dan GitHub Actions sebagai pipeline CI/CD. Dengan demikian, proyek ini tidak hanya menghasilkan aplikasi fungsional, tetapi juga memperlihatkan alur kerja deployment modern yang sesuai dengan konsep cloud computing.

1.4 Manfaat

Manfaat dari proyek ini adalah memberikan pemahaman praktis mengenai hubungan antara aplikasi, image, container, orchestrator, network, dan volume. Selain itu, proyek ini menunjukkan bagaimana otomasi testing dan build membantu menjaga kualitas aplikasi. Dari sisi praktis, penggunaan Docker membuat aplikasi lebih mudah dijalankan ulang, sementara volume menjaga data database tetap tersedia. Hasil akhir proyek ini juga dapat menjadi dasar pengembangan ke arah deployment yang lebih profesional pada layanan cloud atau VPS.

BAB 2 ANALISIS DAN ARSITEKTUR

2.1 Deskripsi Aplikasi

Aplikasi yang dikembangkan pada proyek ini adalah Todo List sederhana yang digunakan untuk mengelola data tugas. Pengguna dapat melihat daftar todo, membuat todo baru, melihat detail todo, memperbarui todo, dan menghapus todo yang sudah tidak diperlukan. Walaupun fiturnya sederhana, aplikasi ini dirancang untuk memperlihatkan implementasi konsep cloud computing secara utuh melalui penggunaan container, database terpisah, health check, dan pipeline otomatis.

2.2 Pengguna Sistem

Pengguna sistem adalah mahasiswa atau pengguna umum yang ingin mencatat dan mengelola daftar tugas. Dalam konteks pengujian, pengguna mengakses aplikasi melalui browser atau HTTP client seperti curl untuk memanggil endpoint API. Sistem kemudian memproses request tersebut melalui container aplikasi dan menyimpan atau membaca data pada database yang berjalan di container terpisah.

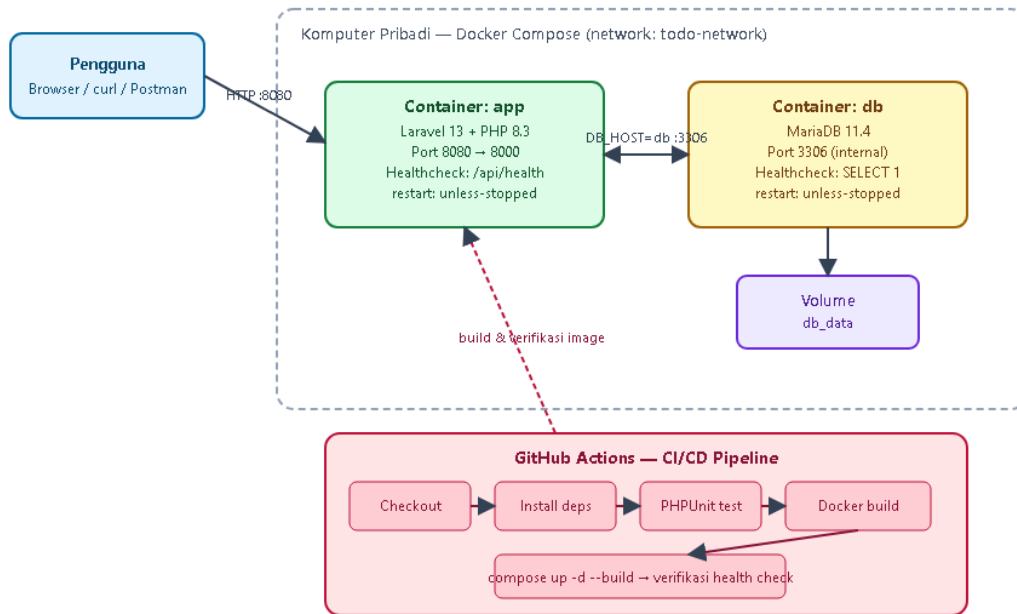
2.3 Teknologi yang Digunakan

Teknologi utama yang digunakan adalah PHP 8.3, Laravel 13, Docker, Docker Compose, MariaDB 11.4, PHPUnit, dan GitHub Actions. Laravel digunakan untuk membangun aplikasi web dan REST API. Docker digunakan untuk containerization, sedangkan Docker Compose digunakan untuk mengorkestrasi beberapa service dalam satu file konfigurasi. MariaDB dipilih sebagai database runtime, sementara SQLite digunakan untuk testing lokal yang terisolasi dari data produksi. GitHub Actions digunakan untuk menjalankan workflow otomatis berupa testing dan build image.

2.4 Diagram Arsitektur

Arsitektur aplikasi terdiri dari pengguna, container aplikasi, container database, network internal, volume penyimpanan data, dan pipeline CI/CD. Pengguna mengakses aplikasi melalui port 8080 pada host. Request diteruskan ke container aplikasi Laravel, lalu aplikasi berkomunikasi dengan database MariaDB melalui network Docker internal bernama todo-network. Data database disimpan pada volume db_data sehingga tetap aman saat container dihentikan atau dibuat ulang. GitHub Actions menjalankan proses checkout, instalasi dependency, automated testing, build image, dan verifikasi health check sebelum image dinyatakan layak. Diagram berikut memperlihatkan bagaimana seluruh komponen tersebut saling terhubung secara deklaratif melalui Docker Compose.

Arsitektur Todo List Application (Docker + Compose + CI/CD)



Sumber: docker-compose.yml & .github/workflows/ci.yml — Todo List Application (Purna Siswantomo & Erifa Dwi Astuti)

Gambar 2.1 Diagram arsitektur aplikasi dan container

2.5 Fungsi Setiap Komponen

Container app berfungsi menjalankan aplikasi Laravel dan menyediakan endpoint API. Container db berfungsi menyimpan data todo secara persisten. Volume db_data menyimpan file database agar data tidak hilang saat container dihapus. Network todo-network memungkinkan container saling berkomunikasi menggunakan nama service, bukan alamat IP statis. GitHub Actions berfungsi menjalankan proses otomatisasi berupa testing dan build. Seluruh komponen tersebut membentuk sistem yang lebih mudah dikelola dibandingkan menjalankan semua bagian secara manual pada satu lingkungan.

BAB 3 IMPLEMENTASI APLIKASI

3.1 Fitur Utama

Aplikasi Todo List memiliki fitur utama CRUD, yaitu create, read, update, dan delete. Pengguna dapat menambahkan data todo baru, melihat semua data todo yang tersimpan, memperbarui isi atau status todo, serta menghapus data yang tidak lagi dibutuhkan. Selain itu, aplikasi memiliki validasi input agar field penting seperti title tidak kosong. Fitur health check juga disediakan untuk memantau apakah aplikasi masih dapat terhubung ke database dengan baik.

3.2 Struktur Data

Data todo disimpan dalam tabel todos. Tabel ini memiliki kolom id, title, description, completed, created_at, dan updated_at. Kolom title menyimpan nama tugas, description menyimpan keterangan tambahan, dan completed menyimpan status tugas yang telah selesai atau belum. Struktur data ini sederhana, tetapi cukup untuk membuktikan kemampuan aplikasi dalam melakukan operasi data dasar yang terhubung ke database.

3.3 Implementasi Model dan Controller

Model Todo digunakan untuk mendefinisikan atribut yang dapat diisi dan mengatur cast boolean pada field completed. Controller TodoController menangani logika index, store, show, update, dan destroy. Setiap request divalidasi sebelum data disimpan atau diubah, sehingga aplikasi memiliki kontrol dasar terhadap input pengguna.

```
public function store(Request $request)
{
    $validated = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'nullable|string',
        'completed' => 'boolean',
    ]);

    $todo = Todo::create($validated);

    return response()->json($todo, 201);
}
```

3.4 Routing dan Endpoint

Routing aplikasi didefinisikan pada routes/api.php. Endpoint /api/todos digunakan untuk akses data todo, sedangkan endpoint /api/health digunakan untuk memeriksa kesehatan layanan dengan cara mencoba membuka koneksi PDO ke database. Route resource digunakan agar endpoint CRUD dapat dibuat secara ringkas.

```
Route::get('/health', function () {
    try {
        DB::connection()->getPdo();
        return response()->json(['status' => 'healthy']);
    } catch (\Exception $e) {
        return response()->json(['status' => 'unhealthy', 'error' => $e->getMessage()],
503);
    }
});

Route::apiResource('todos', TodoController::class);
```

3.5 Konfigurasi Environment

Konfigurasi aplikasi menggunakan file `.env.example` sebagai template environment. File ini berisi parameter seperti `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, dan `DB_ROOT_PASSWORD`. File `.env` yang berisi nilai aktual tidak diunggah ke repository (didaftarkan pada `.gitignore`), sehingga credential tidak pernah tersimpan langsung di riwayat commit. `docker-compose.yml` sendiri hanya mereferensikan variabel tersebut melalui sintaks `${VAR}`, bukan menuliskan nilai kredensial secara eksplisit.

BAB 4 IMPLEMENTASI CONTAINER

4.1 Dockerfile

Dockerfile digunakan untuk membangun image aplikasi Laravel. Base image yang dipakai adalah `php:8.3-cli-alpine` karena ringan dan sesuai untuk menjalankan aplikasi PHP. Sistem dependency dan ekstensi PHP yang dibutuhkan diinstal di dalam image, termasuk `pdo_mysql` untuk koneksi ke MariaDB. Setelah source code disalin, Composer dijalankan untuk memasang dependency aplikasi. Container kemudian dijalankan melalui entrypoint script agar migrasi database dilakukan saat database sudah siap.

```
FROM php:8.3-cli-alpine
RUN apk add --no-cache git curl libpng-dev libxml2-dev zip unzip \
    sqlite-dev oniguruma-dev linux-headers mariadb-client $PHPIZE_DEPS
RUN docker-php-ext-install pdo pdo_sqlite pdo_mysql mbstring exif pcntl bcmath gd xml
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
WORKDIR /var/www
COPY . .
RUN cp .env.example .env
RUN composer install --optimize-autoloader --no-interaction
RUN php artisan key:generate --force
COPY docker/entrypoint.sh /var/www/docker/entrypoint.sh
EXPOSE 8000
CMD ["sh", "/var/www/docker/entrypoint.sh"]
```

4.2 Docker Compose

Docker Compose mengatur dua service utama, yaitu `app` dan `db`. Service `app` membangun image dari Dockerfile, memetakan port 8080 ke port internal 8000, dan mengakses database melalui hostname `db`. Service `db` menggunakan image MariaDB 11.4 dan menyimpan datanya pada volume `db_data`. Kredensial kedua service diambil dari environment variable (`${DB_DATABASE}`, `${DB_USERNAME}`, `${DB_PASSWORD}`, `${DB_ROOT_PASSWORD}`) yang bersumber dari file `.env`, bukan dituliskan langsung di dalam file yang dikirim ke repository.

```
db:
  image: mariadb:11.4
  environment:
    MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
```

```

    MYSQL_DATABASE: ${DB_DATABASE}
    MYSQL_USER: ${DB_USERNAME}
    MYSQL_PASSWORD: ${DB_PASSWORD}
volumes:
  - db_data:/var/lib/mysql
healthcheck:
  test: ["CMD-SHELL", "mariadb ... -u$$MYSQL_USER -p$$MYSQL_PASSWORD ..."]

app:
  build: .
  ports: ["8080:8000"]
  environment:
    DB_HOST: db
    DB_DATABASE: ${DB_DATABASE}
    DB_USERNAME: ${DB_USERNAME}
    DB_PASSWORD: ${DB_PASSWORD}
  depends_on:
    db:
      condition: service_healthy
  restart: unless-stopped

volumes:
  db_data:

```

4.3 Network dan Komunikasi Antar Service

Docker network todo-network digunakan agar service aplikasi dan database dapat saling berkomunikasi di dalam jaringan internal Docker. Aplikasi tidak perlu mengetahui IP database secara statis, cukup menggunakan nama service db sebagai hostname (DB_HOST=db). Cara ini lebih stabil dan sesuai dengan pola deployment container yang umum digunakan. Network internal juga memisahkan komunikasi aplikasi dari akses luar yang tidak diperlukan, karena port database 3306 tidak dipetakan ke host.

4.4 Persistent Volume

Persistent volume diterapkan pada service database melalui volume db_data yang dipetakan ke /var/lib/mysql di dalam container. Volume ini menyimpan data MariaDB di luar siklus hidup container, sehingga data tidak hilang ketika container dihentikan atau dibuat ulang dengan docker compose down lalu docker compose up -d. Penggunaan volume merupakan komponen penting dalam cloud computing karena menjaga ketersediaan data dan mengurangi risiko kehilangan informasi akibat perubahan container. Tanpa volume, database akan bersifat sementara dan tidak cocok untuk skenario penyimpanan data produksi. Bukti pengujian persistent volume disajikan pada Bab 6.4.

4.5 Health Check dan Ketahanan Layanan

Health check ditambahkan pada service database dan aplikasi. Pada database, health check menjalankan query SELECT 1 menggunakan kredensial dari environment variable container (\$\$MYSQL_USER / \$\$MYSQL_PASSWORD) untuk memastikan MariaDB sudah siap menerima koneksi sebelum aplikasi dijalankan (depends_on dengan condition: service_healthy). Pada aplikasi, endpoint

/api/health digunakan untuk memastikan aplikasi dan koneksi database tetap sehat, dan akan mengembalikan status 503 apabila koneksi database gagal. Restart policy unless-stopped juga diterapkan pada kedua service agar layanan dapat kembali aktif jika mengalami gangguan tak terduga. Kombinasi ini membuat sistem lebih tangguh dan mudah dipantau saat demo maupun saat dijalankan berulang kali. Bukti simulasi gangguan layanan disajikan pada Bab 6.5.

BAB 5 IMPLEMENTASI CI/CD

5.1 Tujuan Pipeline

Pipeline CI/CD digunakan untuk memastikan bahwa setiap perubahan kode melewati proses validasi otomatis sebelum dianggap siap digunakan. Pada proyek ini, GitHub Actions digunakan untuk menjalankan testing, build Docker image, dan verifikasi health check. Tujuan utama pipeline adalah menjaga kualitas kode, mengurangi kesalahan manual, dan membuktikan bahwa image yang dibangun dapat dijalankan dengan benar.

5.2 Alur Workflow

Workflow `.github/workflows/ci.yml` dipicu pada setiap push dan pull request ke branch main. Workflow dimulai dengan checkout source code, kemudian setup PHP 8.3 dan instalasi dependency Composer (dengan cache). Setelah itu, file environment disiapkan (`cp .env.example .env`), application key dibuat, dan database SQLite disiapkan khusus untuk pengujian di CI sehingga tidak menyentuh data produksi. Berikutnya, automated testing dijalankan menggunakan PHPUnit. Jika test berhasil, workflow melanjutkan ke proses build Docker image, menyalakan stack Compose penuh (`app + db`), lalu memverifikasi health check dan endpoint yang terhubung ke database sebelum stack dimatikan kembali.

```
jobs:
  test-and-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with: { php-version: '8.3' }
      - run: composer install --no-progress --prefer-dist
      - run: cp .env.example .env && php artisan key:generate --force
      - run: touch database/database.sqlite
      - run: php artisan migrate --force          # DB_CONNECTION=sqlite (testing)
      - run: php artisan test
      - run: docker build -t todo-app:latest .
      - run: docker compose up -d --build
      - run: curl -fsS http://localhost:8080/api/health
      - run: curl -fsS http://localhost:8080/api/todos
      - run: docker compose down -v
```

5.3 Automated Testing sebagai Quality Gate

Automated testing berperan sebagai quality gate sebelum Docker image dinyatakan layak dibangun dan dijalankan. Test yang tersedia mencakup list todo, create todo, show todo, update todo, delete todo, dan validasi title wajib diisi (total 6 test khusus TodoApiTest, ditambah 2 test bawaan Laravel, sehingga 8 test berjalan). Dengan test ini, perubahan pada controller, route, atau validasi dapat terdeteksi lebih cepat sebelum masuk ke tahap build image maupun deployment. Test sengaja dijalankan di atas database SQLite yang terisolasi (bukan database produksi MariaDB), agar proses pengujian tidak berisiko mengubah atau menghapus data nyata milik pengguna.

5.4 Simulasi Pipeline Gagal dan Berhasil

Sebagai bukti penerapan CI/CD, pipeline gagal pernah disimulasikan secara terkontrol melalui perubahan pada test (commit "Simulasi pipeline gagal"), sehingga GitHub Actions menampilkan status merah dan log kesalahan yang jelas. Setelah itu, kesalahan diperbaiki (commit "Memperbaiki test pipeline") dan pipeline dijalankan kembali hingga berhasil dengan status hijau. Bukti pipeline gagal dan berhasil ini penting karena menunjukkan bahwa workflow benar-benar dipakai untuk memverifikasi kualitas codebase, bukan hanya sekadar file konfigurasi yang tidak pernah diuji.

BAB 6 PENGUJIAN

Catatan: pada saat bukti berikut diambil, port host sementara dialihkan dari 8080 ke 8081 karena port 8080 pada komputer pengembang sedang dipakai oleh proyek Docker lain yang tidak berkaitan. Pengalihan ini hanya bersifat sementara untuk pengambilan bukti dan tidak mengubah konfigurasi docker-compose.yml yang dikumpulkan (tetap menggunakan port 8080:8000).

6.1 Pengujian Fungsional

Pengujian fungsional dilakukan untuk memastikan aplikasi dapat menjalankan fitur utama sesuai kebutuhan. Hasil pengujian menunjukkan bahwa aplikasi mampu menampilkan daftar todo, menambahkan todo baru, memperbarui todo, menghapus todo, dan menolak input yang tidak valid. Endpoint /api/health juga berhasil merespons status sehat saat aplikasi dan database berjalan normal.

```
=== docker compose ps (setelah up -d --build) ===
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS
PORTS
todo-app            todo-app-app        "docker-php-entrypoi..."  app        6 minutes ago  Up 6 minutes
(healthy)          0.0.0.0:8081->8000/tcp, [::]:8081->8000/tcp
todo-db             mariadb:11.4        "docker-entrypoint.s..."  db         6 minutes ago  Up 6 minutes
(healthy)          3306/tcp

=== curl http://localhost:8081/api/health ===
{"status":"healthy"}

=== curl http://localhost:8081/api/todos (sebelum tambah data baru) ===
```

```
[{"id":1,"title":"w","description":"web php","completed":false,"created_at":"2026-07-18T09:38:28.000000Z","updated_at":"2026-07-18T09:38:28.000000Z"}]
```

6.2 Pengujian Automated Test

Automated testing dijalankan dengan perintah `php artisan test` di dalam container sekali pakai yang terpisah dari database produksi (menggunakan SQLite khusus testing, meniru langkah yang sama seperti pada pipeline CI). Seluruh 8 test berhasil lulus dengan 18 assertion. Hasil ini menunjukkan bahwa logika aplikasi sudah memiliki perlindungan dasar terhadap regresi tanpa membahayakan data produksi yang sedang berjalan.

```
Container todo-db Running
Container todo-db Waiting
Container todo-db Healthy
Container todo-app-app-run-0959593edce1 Creating
Container todo-app-app-run-0959593edce1 Created

INFO Preparing database.

Creating migration table ..... 9.00ms DONE

INFO Running migrations.

0001_01_01_000000_create_users_table ..... 31.09ms DONE
0001_01_01_000001_create_cache_table ..... 15.62ms DONE
0001_01_01_000002_create_jobs_table ..... 22.49ms DONE
2026_07_05_111029_create_todos_table ..... 3.93ms DONE

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 0.16s

PASS Tests\Feature\TodoApiTest
✓ can list todos 0.17s
✓ can create todo 0.02s
✓ can show todo 0.01s
✓ can update todo 0.01s
✓ can delete todo 0.01s
✓ validates required title 0.01s

Tests: 8 passed (18 assertions)
Duration: 0.49s
```

6.3 Pengujian Container dan Compose

Pengujian container dilakukan dengan menjalankan `docker compose up -d --build` dan memeriksa versi serta status service menggunakan `docker --version`, `docker compose version`, dan `docker`

compose ps. Hasil pengujian menunjukkan bahwa container app dan db dapat berjalan dengan status healthy. Aplikasi dapat diakses melalui port yang dipetakan, dan endpoint API dapat merespons dengan benar.

```
=== docker --version ===
Docker version 29.6.1, build 8900f1d

=== docker compose version ===
Docker Compose version v5.3.0

=== docker compose ps ===
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
todo-app-app:latest			0ded50e66198	869MB	222MB	U
todo-app:test			8e150fba5433	817MB	213MB	
todo-app:v1			d8514ef1284c	690MB	169MB	

6.4 Pengujian Persistent Volume

Pengujian persistent volume dilakukan dengan menambahkan data baru ke database melalui endpoint POST /api/todos, kemudian menjalankan docker compose down (tanpa flag -v, sehingga volume tidak ikut dihapus), lalu menjalankan docker compose up -d kembali. Setelah container hidup kembali, data yang baru ditambahkan tetap tersedia karena disimpan pada volume db_data. Hasil ini membuktikan bahwa volume berfungsi sebagai mekanisme persistensi data, sehingga database tidak kehilangan isi saat lifecycle container berubah.

Data sebelum docker compose down:

```
=== Menambahkan data baru sebelum uji persistent volume ===
{"title":"Bukti persistent volume UAS","description":"Ditambahkan sebelum docker compose down","updated_at":"2026-07-20T19:00:35.000000Z","created_at":"2026-07-20T19:00:35.000000Z","id":9}

=== Data lengkap sebelum down ===
[{"id":9,"title":"Bukti persistent volume UAS","description":"Ditambahkan sebelum docker compose down","completed":false,"created_at":"2026-07-20T19:00:35.000000Z","updated_at":"2026-07-20T19:00:35.000000Z"}, {"id":1,"title":"w","description":"web php","completed":false,"created_at":"2026-07-18T09:38:28.000000Z","updated_at":"2026-07-18T09:38:28.000000Z"}]
```

Proses docker compose down (volume db_data tetap ada):

```
=== docker compose down (tanpa -v, volume db_data tetap ada) ===

=== docker volume ls | grep db_data ===
local      faris_db_data
local      todo-app_db_data
```

Data setelah docker compose up -d kembali (identik dengan sebelum down):

```
=== docker compose ps (setelah up -d kembali) ===
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
todo-app	todo-app-app	"docker-php-entrypoi..."	app	14 seconds ago	Up 8 seconds
(healthy)	0.0.0.0:8081->8000/tcp, [::]:8081->8000/tcp				
todo-db	mariadb:11.4	"docker-entrypoint.s..."	db	14 seconds ago	Up 14 seconds
(healthy)	3306/tcp				

```
=== curl /api/todos (setelah down lalu up kembali) ===
```

```
[{"id":9,"title":"Bukti persistent volume UAS","description":"Ditambahkan sebelum docker compose down","completed":false,"created_at":"2026-07-20T19:00:35.000000Z","updated_at":"2026-07-20T19:00:35.000000Z"}, {"id":1,"title":"w","description":"web php","completed":false,"created_at":"2026-07-18T09:38:28.000000Z","updated_at":"2026-07-18T09:38:28.000000Z"}]
```

6.5 Pengujian Ketahanan Layanan (Health Check)

Ketahanan layanan diuji dengan menghentikan container database secara paksa (docker stop todo-db) untuk mensimulasikan gangguan layanan. Endpoint /api/health pada aplikasi terbukti mendeteksi kegagalan koneksi database dan mengembalikan status unhealthy dengan kode HTTP 503, alih-alih menampilkan error yang tidak terkontrol. Setelah database dinyalakan kembali (docker start todo-db), endpoint /api/health kembali melaporkan status healthy dengan kode HTTP 200 dalam waktu singkat, menunjukkan bahwa layanan mampu pulih tanpa perlu membangun ulang container maupun kehilangan data.

Kondisi saat database dihentikan:

```
=== Kondisi awal: semua sehat ===
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
todo-app	todo-app-app	"docker-php-entrypoi..."	app	28 seconds ago	Up 22 seconds
(healthy)	0.0.0.0:8081->8000/tcp, [::]:8081->8000/tcp				
todo-db	mariadb:11.4	"docker-entrypoint.s..."	db	28 seconds ago	Up 28 seconds
(healthy)	3306/tcp				

```
=== Menghentikan container database (docker stop todo-db) untuk simulasi gangguan ===
todo-db
```

```
=== curl /api/health saat database mati ===
```

```
{"status":"unhealthy","error":"SQLSTATE[HY000] [2002] php_network_getaddresses: getaddrinfo for db failed: Name does not resolve"}
HTTP_STATUS:503
```

Kondisi setelah database dinyalakan kembali:

```
=== docker compose ps saat database berhenti (app health check ikut menurun) ===
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
todo-app	todo-app-app	"docker-php-entrypoi..."	app	About a minute ago	Up About a

```

minute (healthy) 0.0.0.0:8081->8000/tcp, [::]:8081->8000/tcp

=== Menyalakan kembali database: docker start todo-db ===
todo-db

=== docker compose ps setelah database dinyalakan kembali ===
NAME          IMAGE          COMMAND          SERVICE  CREATED          STATUS
PORTS
todo-app      todo-app-app   "docker-php-entryp...  app      About a minute ago Up About a
minute (healthy) 0.0.0.0:8081->8000/tcp, [::]:8081->8000/tcp
todo-db       mariadb:11.4   "docker-entrypoint.s..." db        About a minute ago Up 15
seconds (healthy) 3306/tcp

=== curl /api/health setelah database pulih ===
{"status":"healthy"}
HTTP_STATUS:200

```

BAB 7 KESIMPULAN

7.1 Kesimpulan Umum

Proyek Todo List ini berhasil mengimplementasikan aplikasi sederhana dalam arsitektur multi-container menggunakan Docker, Docker Compose, MariaDB, health check, persistent volume, automated testing, dan GitHub Actions. Walaupun aplikasi yang dikembangkan tidak kompleks, struktur implementasinya sudah mencerminkan praktik dasar cloud computing yang baik, terutama pada aspek containerization, orchestration, dan automation. Dengan demikian, proyek ini memenuhi fokus utama UAS, yaitu penerapan konsep cloud computing secara utuh dan dapat dibuktikan.

7.2 Kendala dan Solusi

Kendala utama pada awal pengembangan adalah penggunaan database SQLite tunggal yang belum memenuhi syarat multi-container. Kendala tersebut diselesaikan dengan memindahkan runtime database ke MariaDB dalam container terpisah dan menambahkan volume untuk persistensi data. Kendala lain adalah memastikan aplikasi tidak start sebelum database siap, sehingga ditambahkan health check dan entrypoint script yang menunggu migrasi berhasil. Ditemukan pula bahwa menjalankan automated test langsung di dalam container produksi yang sedang berjalan berisiko mencampur data uji dengan data nyata; solusinya adalah menjalankan test pada container sekali pakai (docker compose run) dengan koneksi SQLite terpisah, sama seperti yang dilakukan pipeline CI. Kredensial database yang semula dituliskan langsung di docker-compose.yml juga dipindahkan menjadi environment variable yang bersumber dari file .env agar tidak tersimpan sebagai plaintext di repository.

7.3 Rencana Pengembangan

Jika proyek ini dikembangkan lebih lanjut, service tambahan seperti Redis, Nginx, atau phpMyAdmin dapat ditambahkan untuk memperkaya arsitektur. Selain itu, backup database, logging terpusat, dan publikasi image ke Docker Hub atau GitHub Container Registry juga dapat diterapkan sebagai nilai

tambah. Dalam skenario cloud, aplikasi ini juga dapat dipindahkan ke VPS atau layanan cloud dengan perubahan konfigurasi yang relatif kecil karena arsitekturnya sudah berbasis container; langkah yang diperlukan umumnya berupa penyesuaian DNS/reverse proxy, penggantian volume lokal dengan managed storage, dan penambahan secret management pada level platform cloud yang dituju.

LAMPIRAN

Bukti eksekusi lengkap (raw output) tersedia pada folder evidence/ di repository:

evidence/01_docker_version_and_ps.txt - docker --version, docker compose version, docker compose ps

evidence/02_stack_running_and_health.txt - stack berjalan healthy, /api/health, /api/todos

evidence/03_php_artisan_test.txt - hasil php artisan test (8 test, isolated SQLite)

evidence/04_persistence_before_down.txt - data sebelum docker compose down

evidence/05_compose_down.txt - docker compose down, volume db_data tetap ada

evidence/06_persistence_after_up.txt - data setelah docker compose up -d kembali

evidence/07_resilience_db_down.txt - simulasi database mati, /api/health -> 503 unhealthy

evidence/08_resilience_recovery.txt - database pulih, /api/health -> 200 healthy

evidence/09_docker_images.txt - docker images (image todo-app-app berhasil dibangun)

docs/architecture-diagram.svg / .png - diagram arsitektur (sumber vektor dan raster)

Format Pengumpulan

Nama/NIM/Kelas: Purna Siswantomo / C2C023160 / [Kelas Anda]

Nama/NIM/Kelas: Erifa Dwi Astuti / C2C023161 / [Kelas Anda]

Nama aplikasi: Todo List Application

Link repository GitHub: <https://github.com/Purna-Siswantomo/ToDo-docker>

Link pipeline gagal: <https://github.com/Purna-Siswantomo/ToDo-docker/actions/runs/28741426116>

Link pipeline berhasil: <https://github.com/Purna-Siswantomo/ToDo-docker/actions/runs/28741450472>

Link video demonstrasi: [diisi setelah video direkam dan diunggah]

Link image registry (opsional): [belum tersedia]

Link aplikasi online (opsional): [belum tersedia]